SnapPlate

Snap a meal — know what’s in it.

Technical Documentation

A multimodal meal-analysis web application built on Claude vision.

Project	SnapPlate
Version	0.1.0
Document date	28 June 2026
Stack	Next.js 16 · React 19 · TypeScript · Claude (claude-opus-4-8) vision
Derived from	PS-1 Project List, item #10 — AI-Based Dietary Assessment Using Multimodal LLMs

Contents

1. Introduction
2. Origin and Problem Statement
3. Features
4. How It Works (Pipeline)
5. Architecture
6. Technology Stack
7. Project Structure
8. Data Model and JSON Schema
9. API Reference
10. Demo Mode
11. Setup and Installation
12. Usage Guide
13. Security and Privacy
14. Limitations and Disclaimers
15. Verification Status
16. Future Enhancements

Appendix A. Sample Analysis Output

1. Introduction

SnapPlate is a web application that turns a single photo of a meal into an instant, structured nutritional breakdown. The user points a camera (or uploads an image) at any plate of food; the app returns each identified food item, an estimated portion, per-item and total calories and macronutrients (protein, carbohydrate, fat, fibre), an overall health score from 0 to 100, and one specific, actionable suggestion to make that exact meal healthier. Analysed meals can be saved to a per-day log kept locally in the browser.

The core engine is a multimodal large language model — Claude (`claude-opus-4-8`) — invoked with the meal image and a constrained JSON output schema, so every response conforms to a known shape. No manual food logging, calorie database lookup, or barcode scanning is involved.

1.1 Design goals

- **Zero-friction capture.** One tap from camera to result; no forms.
- **Trustworthy structure.** The model is constrained to a strict JSON schema, so the UI never has to parse free text.
- **Actionable, not just descriptive.** Every result ends with one concrete way to improve that specific meal.
- **Runs out of the box.** A built-in demo mode means the app is fully usable without any API key or external account.
- **Private by default.** The daily log lives only in the browser; the API key never reaches the client.

2. Origin and Problem Statement

SnapPlate was derived from the NGIT/KMEC Project School “PS-1” project list, specifically item #10: *AI-Based Dietary Assessment Using Multimodal Large Language Models*. The original brief notes that traditional dietary assessment relies on manual food logs and expert analysis, which are slow and inaccurate; when a user uploads a photo of a meal or describes it, existing systems struggle to identify the food items, portion sizes, and nutrient values reliably.

SnapPlate keeps the core idea — a multimodal LLM turning an everyday photo into structured, actionable insight — and reframes it from an academic assessment tool into a consumer-facing health-coach application with a clean capture-to-result flow and a running daily log.

2.1 Problem statement

People who want to eat more mindfully rarely log meals consistently, because manual entry is tedious and portion estimation is hard. There is a need for a fast, low-effort way to understand the approximate nutritional content of a real meal from a photo, together with a single clear next step to improve it.

3. Features

- **Photo capture or upload** — on mobile, the camera opens directly; on desktop, drag-and-drop or file picker.

- **Per-item recognition** — each distinct food/drink is listed with an estimated portion (e.g. “1 cup”, “~150g”).
- **Calorie and macro breakdown** — per item and as a meal total, including fibre.
- **Health score** — a single 0–100 figure reflecting overall nutritional quality.
- **Make-it-healthier tip** — one specific, actionable suggestion tailored to the analysed meal.
- **Confidence flag** — low / medium / high, reflecting image clarity and portion ambiguity.
- **Optional context note** — the user can add a hint the photo cannot show (e.g. “cooked in olive oil”).
- **Daily log** — saved meals and a running calorie total for the day, persisted in the browser via localStorage.
- **Non-food detection** — if the image is not food, the app says so instead of inventing numbers.
- **Demo mode** — with no API key set, the app returns clearly labelled sample results so the entire flow is usable immediately.

4. How It Works (Pipeline)

A single analysis follows the steps below.

Step	Where	What happens
1. Capture	Browser	User selects or photographs a meal.
2. Down scale	Browser	The image is resized to a maximum 1024 px long edge and re-encoded as JPEG via a canvas, keeping the upload small and reducing image-token cost.
3. Request	Browser → Server	Base64 image, media type, and any note are POSTed to `/api/analyze`.
4. Analyse	Server → Claude	The route sends the image to Claude with a dietitian system prompt and a JSON-schema output constraint.
5. Return	Server → Browser	A validated JSON analysis object is returned.
6. Render	Browser	The breakdown, score, and tip are displayed.
7. Log	Browser	The user may add the meal to the day’s localStorage log.

5. Architecture

SnapPlate is a single Next.js (App Router) application with a thin server boundary. The browser handles capture, downscaling, rendering, and local persistence; a single server route handler holds the API key and talks to Claude. There is no database.

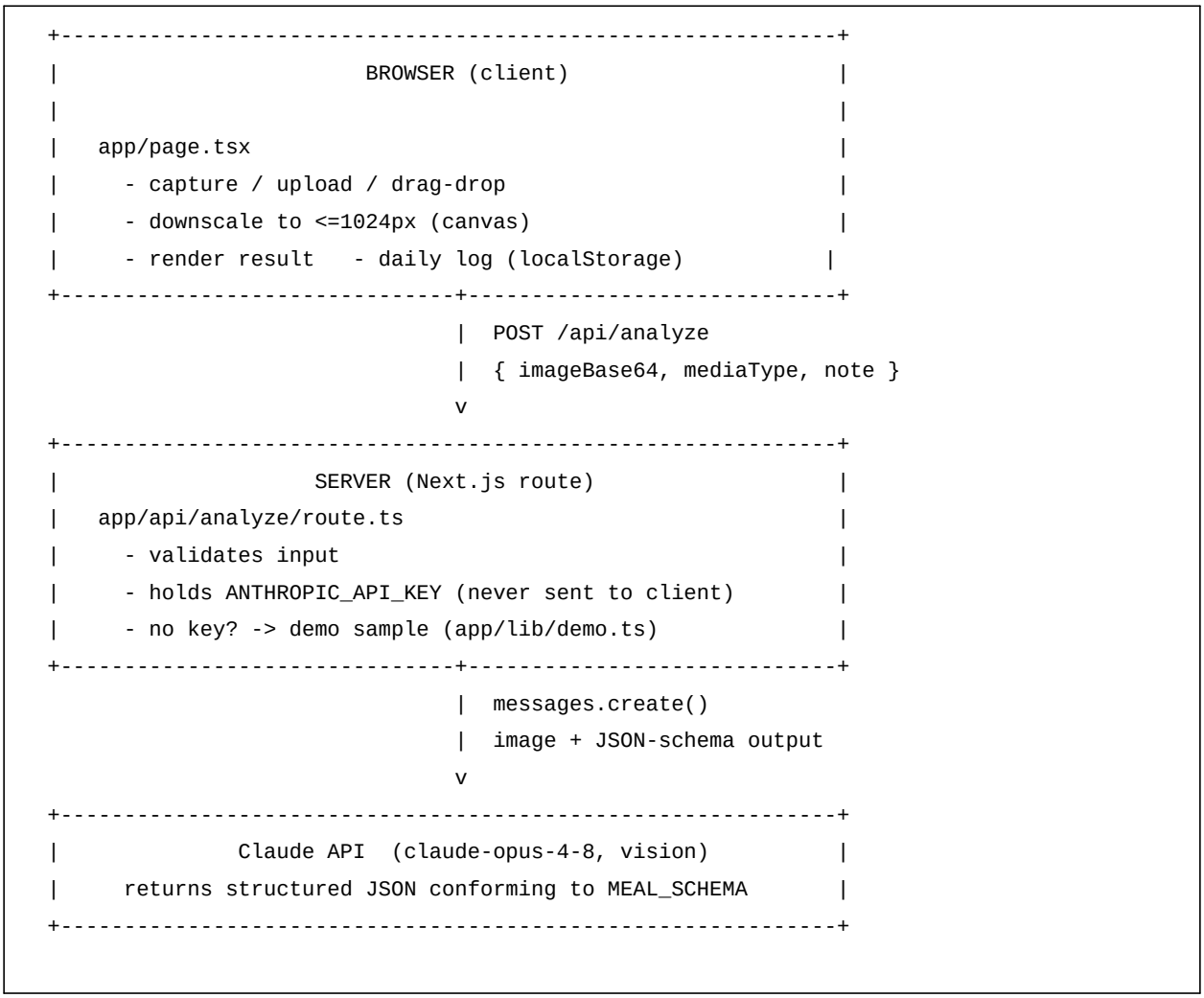

Figure 1. End-to-end data flow. The API key is confined to the server route; the daily log never leaves the browser.

6. Technology Stack

Layer	Choice	Notes
Framework	Next.js 16 (App Router)	Server route handler + static client page.
UI	React 19	Single client component; no UI library.
Language	TypeScript 5	Strict mode; shared types across client and server.
AI model	Claude claude-opus-4-8	Multimodal vision with structured (JSON-schema) output.
SDK	@anthropic-ai/sdk ≥ 0.106	Required for the output_config structured-output API.
Styling	Plain CSS	Single global stylesheet; no Tailwind/CSS-in-JS.
Persistence	Browser localStorage	Per-day meal log; no server database.
Runtime	Node.js 22	Route handler runs on the Node runtime.

7. Project Structure

```
snapplate/
  app/
    api/
      analyze/
        route.ts        # server: calls Claude, returns JSON (or demo)
    lib/
      meal.ts           # shared types + MEAL_SCHEMA (JSON schema)
      demo.ts           # canned sample analyses for no-key demo mode
    layout.tsx          # root layout + metadata
    page.tsx            # client UI: capture, results, daily log
    globals.css         # all styling (black/white-friendly tokens)
  next.config.mjs       # Next config (turbopack root pinned)
  tsconfig.json
  package.json
  .env.local.example    # template for ANTHROPIC_API_KEY
  README.md
```

8. Data Model and JSON Schema

All shapes are defined once in `app/lib/meal.ts` and shared by the client and server. The same schema is handed to Claude as a structured-output constraint, so the model is forced to return exactly this shape.

8.1 MealAnalysis

Field	Type	Description
`is_food`	boolean	True only if the image actually shows food or drink.
`dish_name`	string	Short name for the meal as a whole.
`items`	MealItem[]	One entry per identified food/drink item.
`total`	MealTotals	Summed calories, macros, and fibre for the meal.
`health_score`	integer 0–100	Overall nutritional quality (range enforced by prompt).
`healthier_tip`	string	One specific, actionable improvement for this meal.
`confidence`	"low" | "medium" | "high"	Estimate confidence given clarity and portion ambiguity.
`demo`	boolean (optional)	Present and true only for no-key demo-mode results.

8.2 MealItem

Field	Type	Description
`name`	string	Food/drink item name.
`estimated_portion`	string	Human-readable portion, e.g. "1 cup", "~150g".
`calories`	number	Estimated kcal for the item.

Field	Type	Description
`protein_g / carbs_g / fat_g`	number	Estimated macronutrients in grams.

8.3 MealTotals

Object with numeric fields `calories`, `protein_g`, `carbs_g`, `fat_g`, and `fiber_g`.

8.4 LoggedMeal

A `MealAnalysis` extended with `id` (string) and `loggedAt` (ISO timestamp). This is the shape stored in localStorage under the key `snapplate.log.v1`.

Schema note: the structured-output API does not support numeric range or string-length constraints, so the 0–100 bound on `health_score` is enforced through the system prompt rather than the schema. Every object additionally sets `additionalProperties: false` with an explicit `required` list.

9. API Reference

9.1 POST /api/analyze

Analyses one meal image. Runs on the Node runtime with a 60-second maximum duration.

Request body (application/json):

```
{
  "imageBase64": "",
  "mediaType":   "image/jpeg",   // or png / webp / gif
  "note":        "optional free-text context"
}
```

Successful response (200): a MealAnalysis object (see Section 8). In demo mode the same shape is returned with `demo: true`.

Error responses:

Status	Condition
400	Invalid body, missing image data, or unsupported media type.
422	The model declined to analyse the image (refusal).
502	Upstream Claude API error or no analysis returned.
500	Unexpected server error.

Note: a missing API key is *not* an error — it triggers demo mode and returns 200. Supported media types are JPEG, PNG, WebP, and GIF.

10. Demo Mode

Demo mode makes the application fully usable with zero configuration. When the server route detects that `ANTHROPIC_API_KEY` is not set, it skips the Claude call and returns one of several canned sample analyses defined in `app/lib/demo.ts` (for example, a grilled chicken bowl, a cheeseburger with fries,

and a vegetarian thali), chosen at random.

- Every demo result carries `demo: true`.
- The UI shows a clearly worded banner stating the result is sample data and is not based on the uploaded photo.
- The full flow — upload, analyse, view, and add to the daily log — works identically to live mode.

This keeps the demonstration honest: the sample numbers are never presented as a real reading of the user's image. Adding a valid API key and restarting switches the app to live analysis and the banner disappears.

11. Setup and Installation

Prerequisites: Node.js 22 or newer and npm.

11.1 Run without a key (demo mode)

```
cd snapplate
npm install
npm run dev      # http://localhost:3000 - works immediately
```

11.2 Enable live analysis

```
cp .env.local.example .env.local
# edit .env.local and set:
#   ANTHROPIC_API_KEY=sk-ant-...
npm run dev      # restart; demo banner disappears
```

An API key can be created from the Anthropic Console. New accounts typically receive some free trial credit, which is sufficient for testing live analysis.

11.3 Production build

```
npm run build
npm run start
```

12. Usage Guide

- Open the app and tap the capture area. On a phone this opens the camera; on desktop it opens a file picker (you can also drag an image in).
- Optionally type a short note for anything the photo cannot show.
- Tap **Analyze meal** and wait for the result.
- Review the dish name, health score, macro tiles, per-item list, and the make-it-healthier tip.
- Tap **Add to today's log** to save it, or **Discard** to drop it.

- The daily log shows each saved meal with its time and calories, plus a running daily total. Remove any entry with the × control.

13. Security and Privacy

- **API key isolation.** The Anthropic API key is read only on the server route and is never included in any client response or bundle.
- **Local-only log.** The meal log is stored in the browser’s localStorage. There is no account system and no server-side database, so logged meals never leave the user’s device.
- **Image handling.** The meal image is sent to the server only to be forwarded to the Claude API for analysis; the application does not persist uploaded images.
- **Input validation.** The route validates the request body and rejects unsupported media types before making any upstream call.

14. Limitations and Disclaimers

- Nutritional figures are AI-generated estimates from a single photo and are approximate. They are **not** medical, clinical, or dietary advice.
- Portion estimation from a 2-D image is inherently uncertain; the confidence flag reflects this.
- Hidden ingredients (oils, sugar, sauces) may be under- or over-counted; the optional note partly mitigates this.
- Demo-mode results are fixed samples and bear no relation to the uploaded image.
- The daily log is per-browser and per-device; clearing browser storage erases it.

15. Verification Status

Check	Result
Production build (npm run build)	Passes; TypeScript type-check clean.
Dev server boot	Starts; home page returns HTTP 200.
/api/analyze in demo mode	Returns a complete sample MealAnalysis (200).
Live Claude analysis	Not yet verified — requires an API key in .env.local.

16. Future Enhancements

- Weekly trends with a calorie/macro chart over time.
- Per-user goals (target kcal/protein) with progress indicators.
- Optional sign-in and a database so the log syncs across devices.

- A cheaper/faster model option for high-volume use.
- Barcode or label scanning to complement photo analysis.

Appendix A. Sample Analysis Output

A representative `MealAnalysis` response:

```
{
  "is_food": true,
  "dish_name": "Grilled chicken bowl with rice & veg",
  "items": [
    { "name": "Grilled chicken breast",
      "estimated_portion": "~150g",
      "calories": 248, "protein_g": 46,
      "carbs_g": 0, "fat_g": 6 },
    { "name": "Steamed white rice",
      "estimated_portion": "1 cup",
      "calories": 205, "protein_g": 4,
      "carbs_g": 45, "fat_g": 0 },
    { "name": "Mixed vegetables",
      "estimated_portion": "1 cup",
      "calories": 80, "protein_g": 3,
      "carbs_g": 16, "fat_g": 1 }
  ],
  "total": { "calories": 533, "protein_g": 53,
             "carbs_g": 61, "fat_g": 7, "fiber_g": 6 },
  "health_score": 78,
  "healthier_tip": "Swap the white rice for brown
     rice to roughly double the fibre.",
  "confidence": "medium"
}
```

End of document.